

Kỹ Thuật Lập Trình Python

Python Object-Oriented Programming



1

CONTENT



1. Class

1. Định nghĩa class
2. Class object
3. Class và Instance variables
4. Instance object
5. Method object

2. Kế thừa (*Inheritance*)

3. Đa hình (*Polymorphism*)

4. Đóng gói (*Encapsulation*)

5. Trừu tượng (*Abstraction*)

Tham khảo từ: <https://docs.python.org/3/tutorial/classes.html>

2

2



Tại sao vận dụng OOP?

- Giải quyết các bài toán thực tế hiệu quả hơn.
- Có thể chia sẻ các lớp (class) nên mã nguồn tái sử dụng.
- Năng suất của chương trình tăng cao.
- Dữ liệu an toàn và bảo mật (trừu tượng).
- ...



Ý tưởng

Viết chương trình tính tiền lương nhân viên (NV) cho một công ty. Công ty gồm 2 loại NV:

1. Nhân viên văn phòng

- Mã nhân viên
- Họ và tên
- Lương cơ bản
- Số ngày làm việc
- Lương hằng tháng

2. Nhân viên bán hàng

- Mã nhân viên
- Họ và tên
- Lương cơ bản
- Số sản phẩm bán được
- Lương hằng tháng

Lương hằng tháng tính bằng công thức:

- NV văn phòng = lương cơ bản + số ngày làm việc * 150_000đ
- NV bán hàng = lương cơ bản + số sản phẩm * 18_000đ

Chương trình đáp ứng các yêu cầu sau:

1. Khởi tạo dữ liệu các nhân viên
2. In các nhân viên: mã nhân viên, họ tên, lương cơ bản, lương hằng tháng
3. Tính lương các nhân viên
4. Tìm nhân viên theo mã nhân viên
5. Tìm lương hằng tháng cao nhất
6. Cập nhật lương cơ bản theo mã NV
7. Cập nhật số ngày làm của nhân viên văn phòng theo mã nhân viên.
8. Tính tổng tiền lương hằng tháng mà công ty phải trả cho các nhân viên.
9. Tìm nhân viên có lương cơ bản cao nhất
10. Tìm nhân viên bán hàng có lương hằng tháng thấp nhất
11. Sắp xếp NV giảm dần theo lương cơ bản.



Giới thiệu OOP

Object-Oriented Programming (OOP)

- Bài toán sẽ được phân tích thành các **đối tượng** mang thuộc tính và hành vi.
- OOP là phương pháp tư duy gần với thực tế cuộc sống của con người
- Giải quyết bài toán dựa vào mối quan hệ giữa các đối tượng tham gia vào bài toán.



Thuộc tính
~ Property
~ Attribute

Hành vi
~ Method
~ Behavior

Kỹ thuật Lập trình Python

8

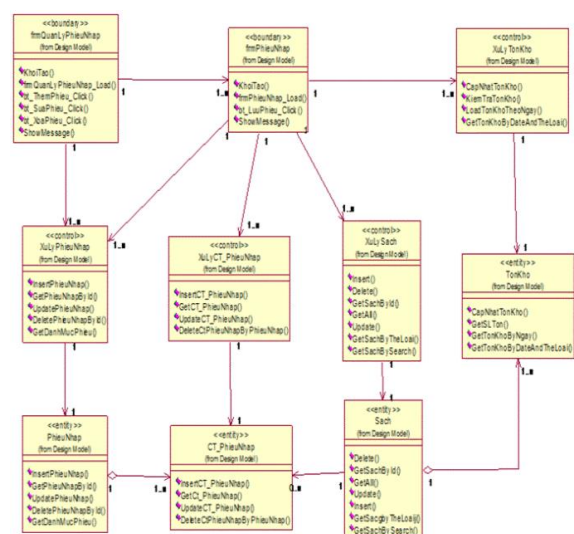
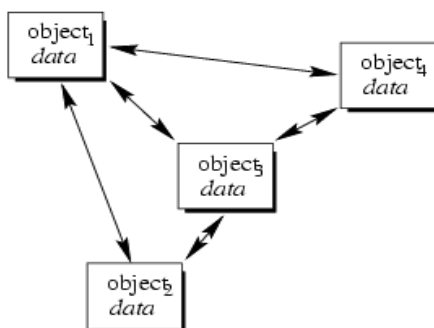
8



Giới thiệu OOP

Object-Oriented Programming (OOP)

- Phương pháp lập trình dựa trên:
 - kiến trúc lớp (class)
 - và đối tượng (object)



Kỹ thuật Lập trình Python

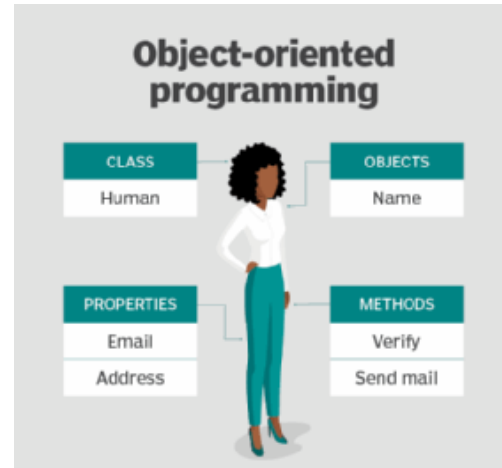
9

9



Các khái niệm

- Đối tượng - Object
- Lớp - Class
- Thuộc tính - Property
- Phương thức - Method



Kỹ thuật Lập trình Python

10

10



Đối tượng

- Đối tượng (object) là một **thực thể** tham gia vào bài toán, có các **thuộc tính dữ liệu** và **hành vi xử lý**.
- Ví dụ: nhân viên, sinh viên, xe...

Object	Property	Method
	ma_nv ho_ten luong_cb so_ng luong_ht	hien_thi_tt() dang_ky() xet_thuong() tinh_luong_ht()

Kỹ thuật Lập trình Python

11

11



Sơ đồ lớp

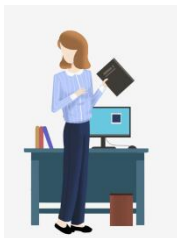
Giới thiệu

- Hỗ trợ biểu diễn cấu trúc của một hệ thống dựa trên các lớp.
- Trình bày mối quan hệ giữa các lớp.
- Biểu đồ mô tả lớp, thuộc tính, phương thức và mối quan hệ giữa các đối tượng trong hệ thống.
- Sơ đồ lớp giúp cho lập trình viên tổng quan về cấu trúc của hệ thống, hỗ trợ quá trình thiết kế, triển khai và bảo trì theo mô hình OOP.
- Có rất nhiều **công cụ hỗ trợ vẽ và tạo code**.
- Khóa học này dùng StarUML.

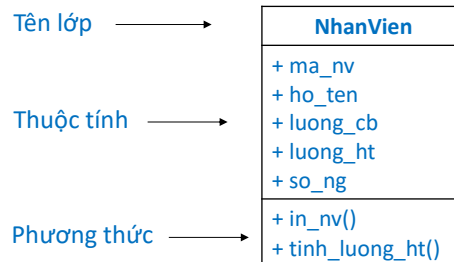


Sơ đồ lớp - Class Diagram

- Mô tả cấu trúc đối tượng



Đối tượng



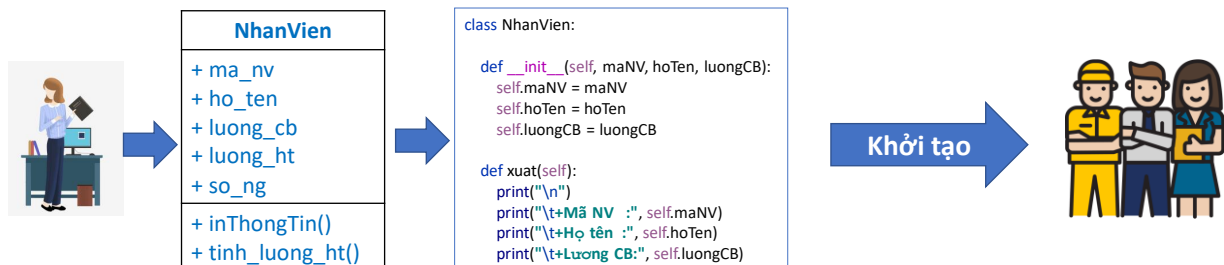
Sơ đồ lớp



Lớp - Class

Giới thiệu

- Class là một bản thiết kế được dùng để **khởi tạo** đối tượng trong lập trình.
- Một class gồm có
 1. Thuộc tính
 2. Phương thức
- Class thực chất là một **kiểu dữ liệu** do người dùng định nghĩa.



Kỹ thuật Lập trình Python

14

14



Lớp - Class

Cú pháp

```
class ClassName:
    <statement-1>
    .
    .
    .
    <statement-N>
```

NhanVien
+ ma_nv
+ ho_ten
+ luong_cb
+ luong_ht
+ so_ng
+ in_nv()
+ tinh_luong_ht()

Khuyến nghị đặt tên theo quy ước PEP 8

Kỹ thuật Lập trình Python

15

15



Lớp - Class

Cú pháp

```
class ClassName:
    <statement-1>
    .
    .
    .
    <statement-N>
```



```
class ClassName:
    <data attribute>
    .
    <constructor>
    .
    .
    .
    .....
    <method>
    .
    .
    .
```

Kỹ thuật Lập trình Python

16

16



Lớp - Class

Ví dụ

NhanVien
+ ma_nv
+ ho_ten
+ luong_cb
+ luong_ht
+ so_ng
+ in_nv()
+ tinh_luong_ht()

```
class NhanVien:
    so_nv = 0

    def __init__(self, ma_nv, ho_ten, luong_cb):
        self.ma_nv = ma_nv
        self.ho_ten = ho_ten
        self.luong_cb = luong_cb
        self.so_ng = so_ng
        self.luong_ht = luong_ht

    def in_nv(self):
        print("\n")
        print("\t+Mã NV   :", self.ma_nv)
        print("\t+Họ tên   :", self.ho_ten)
        print("\t+Lương CB:", self.luong_cb)
        print("\t+Lương HT:", self.luong_ht)
```

```
class NhanVien:

    def __init__(self, ma_nv, ho_ten, luong_cb):
        self.ma_nv = ma_nv
        self.ho_ten = ho_ten
        self.luong_cb = luong_cb
        self.so_ng = so_ng
        self.luong_ht = luong_ht

    def in_nv(self):
        print("\n")
        print("\t+Mã NV   :", self.ma_nv)
        print("\t+Họ tên   :", self.ho_ten)
        print("\t+Lương CB:", self.luong_cb)
        print("\t+Lương HT:", self.luong_ht)
```

Kỹ thuật Lập trình Python

17

17



Thuộc tính - Attribute

Có hai loại thuộc tính

• Thuộc tính của đối tượng - Instance Attribute

- Là thuộc tính gắn liền với **một đối tượng** của lớp, thể hiện đặc điểm của đối tượng.
- Thuộc tính của đối tượng được định nghĩa trong hàm khởi tạo (constructor).
- Truy cập thuộc tính qua **đối tượng**.

• Thuộc tính của lớp - Class Attribute

- Là các biến được khai báo **trực tiếp** trong lớp, được chia sẻ bởi **tất cả các đối tượng** của lớp.
- Truy cập thuộc tính bằng **tên lớp** hoặc gọi từ **đối tượng**.



Thuộc tính - Attribute

Khai báo thuộc tính

Loại	Cú pháp
Thuộc tính của đối tượng	self .ten_thuoc_tinh = <giá trị khởi tạo>
Thuộc tính của lớp	ten_thuoc_tinh = <gán giá trị>

Khuyến nghị đặt tên theo quy ước PEP 8



Thuộc tính - Attribute

Ví dụ

NhanVien
+ ma_nv
+ ho_ten
+ luong_cb
+ luong_ht
+ so_ng
+ in_nv()
+ tinh_luong_ht()

```
class NhanVien:
    so_nv = 0

    def __init__(self):
        NhanVien.so_nv += 1
```

```
class NhanVien:
    def __init__(self, ma_nv, ho_ten, luong_cb, so_ng):
        self.ma_nv = ma_nv
        self.ho_ten = ho_ten
        self.luong_cb = luong_cb
        self.so_ng = so_ng
        self.luong_ht = 0
```

Kỹ thuật Lập trình Python

20

20



Phương thức khởi tạo - Constructor

Constructor là gì?

- Constructor là một phương thức đặc biệt dùng để khởi tạo một đối tượng.
- Phương thức khởi tạo dùng để tạo giá trị cho thuộc tính.
- Không bắt buộc phải có phương thức khởi tạo trong class.

Phương pháp

- Định nghĩa lại phương thức `__init__`:

```
def __init__(self, .....):
    self.ten_thuoc_tinh = <giá trị thuộc tính>
    ....
    ....
```

Kỹ thuật Lập trình Python

21

21



Phương thức khởi tạo - Constructor

Phương thức `__init__()`

- Dùng phương thức `__init__()` để khởi tạo một đối tượng. Ví dụ:

```
def __init__(self):
    self.data = []
```

```
def __init__(self, *arg):
    self.a = ...
```

```
def __init__(self, a, b):
    self.a = a
    self.b = b
```

```
def __init__(self, **kwargs):
    self.a = ...
    self.b = ...
```

```
def __init__(self, a=1, b=2):
    self.a = a
    self.b = b
```

```
def __init__(self, *arg, **kwargs):
    self.a = ...
    self.b = ...
```

Kỹ thuật Lập trình Python

22

22



Phương thức khởi tạo - Constructor

Phương thức `__init__()`

- Ví dụ tạo class biểu diễn phân số:

```
class PhanSo:
    def __init__(self, tu, mau):
        self.tu = tu
        self.mau = mau
```

```
x = PhanSo(3, 5)
```

```
x.tu, x.mau #Kết quả(3, 5)
```

Kỹ thuật Lập trình Python

23

23



Phương thức khởi tạo - Constructor

Phương thức `__init__()`

- Ví dụ:

```
class Complex:
    def __init__(self, realpart, imagpart):
        self.r = realpart
        self.i = imagpart

x = Complex(3.0, -4.5)
x.r, x.i (3.0, -4.5)
```

Kỹ thuật Lập trình Python

24

24



Phương thức khởi tạo - Constructor

Phương thức `__init__()`

- Ví dụ tạo class biểu diễn tọa độ:

```
class ToaDo:
    def __init__(self, x = 0, y = 0):
        self.x = x
        self.y = y

p = ToaDo(3.67, 4.89)
p.x, p.y #Kết quả(3.67, 4.89)
```

Kỹ thuật Lập trình Python

25

25



Phương thức khởi tạo - Constructor

Ví dụ

NhanVien
+ ma_nv
+ ho_ten
+ luong_cb
+ luong_ht
+ so_ng
+ in_nv()
+ tinh_luong_ht()

```
class NhanVien:
    so_nv = 0

    def __init__(self, ma_nv, ho_ten, luong_cb, so_ng):
        self.ma_nv = maNV
        self.ho_ten = ho_ten
        self.luong_cb = luong_cb
        self.so_ng = so_ng
        self.luong_ht = 0

    NhanVien.so_nv += 1
```



Phương thức khởi tạo - Constructor

Nạp chồng khởi tạo – Constructor overloading

- Trong các ngôn ngữ khác, constructor overloading là một kỹ thuật cho phép một class có thể tạo ra các đối tượng bằng nhiều constructor khác nhau.
- Trong Python có hay không?
- Liên tưởng phương pháp tạo constructor bằng cách:
 - Tạo giá trị mặc định, kwargs
 - Kỹ thuật *args
 - Kỹ thuật **kwargs



Phương thức khởi tạo - Constructor

Nạp chồng khởi tạo – Constructor overloading

• Ví dụ

```
def __init__(self):
    self.data = []
```

```
def __init__(self, *arg):
    self.a = ...
```

```
def __init__(self, a, b):
    self.a = a
    self.b = b
```

```
def __init__(self, **kwargs):
    self.a = ...
    self.b = ...
```

```
def __init__(self, a=1, b=2):
    self.a = a
    self.b = b
```

```
def __init__(self, *arg, *kwargs):
    self.a = ...
    self.b = ...
```

Kỹ thuật Lập trình Python

28

28



Phương thức khởi tạo - Constructor

Bài tập

- Dùng kỹ **kwargs để xây dựng phương thức khởi tạo cho class NhanVien.
- Trong đó: maNV bắt buộc, còn lại tùy biến.

```
class NhanVien:
```

```
    def __init__(self, ma_nv:int, **kwargs):
        self.ma_nv = ma_nv
        self.ho_ten = ...
        self.luong_cb = ...
        self.so_ng = ...
        self.luong_ht = ...
```

Kỹ thuật Lập trình Python

29

29



Phương thức thành viên – Instance method

Giới thiệu

- Phương thức thể hiện **hành vi** của đối tượng (thể hiện hành động).

Ví dụ về phương thức:

- Phương thức **tinhDTB(self, ...)** thực hiện tính điểm trung bình cho các môn học.
- Phương thức **tinhLuong(self, ...)** thực hiện tính lương nhân viên.



Phương thức - Method

Cú pháp khai báo phương thức

```
def ten_phuong_thuc(self, <các khai báo input>):
    """docstring: Mô tả hàm"""
    #khởi lệnh
    ...
```

Khuyến nghị đặt tên theo quy ước PEP 8



Phương thức - Method

Ví dụ

NhanVien
+ ma_nv
+ ho_ten
+ luong_cb
+ luong_ht
+ so_ng
+ in_nv()
+ tinh_luong_ht()

class NhanVien:

```
def __init__(self, ma_nv, ho_ten, luong_cb, so_ng):
    self.ma_nv = maNV
    self.ho_ten = ho_ten
    self.luong_cb = luong_cb
    self.so_ng = so_ng
    self.luong_ht = 0
```

```
def in_nv(self):
    print("\n")
    print("\t+Mã NV :", self.ma_nv)
    print("\t+Họ tên :", self.ho_ten)
    print("\t+Lương CB:", self.luong_cb)
```

Kỹ thuật Lập trình Python

32

32



Phương thức - Method

Phương thức truy vấn

- Phương thức truy vấn không làm thay đổi trạng thái hiện tại của đối tượng
- Sử dụng tiền tố "get"<tên của thuộc tính cần truy vấn>
- Phương thức truy vấn điều kiện nên dùng tiền tố "is"
- Ví dụ:

```
def get_x(self):
    return self.x
```

Kỹ thuật Lập trình Python

33

33



Phương thức - Method

Phương thức cập nhật

- Thay đổi trạng thái của đối tượng bằng cách sửa đổi một hoặc nhiều thành viên dữ liệu của đối tượng đó.
- Sử dụng tiền tố **"set"** <tên thuộc tính cần sửa>.
- Ví dụ:

```
def set_x(self, x):
    self.x = x
```

Kỹ thuật Lập trình Python

34

34



Phương thức - Method

Ví dụ

```
class ToaDo:
    def __init__(self, x, y) -> None:
        self.x = x
        self.y = y

    def get_x(self) -> int:
        return self.x

    def set_x(self, x) -> None:
        self.x = x

    def xuất(self) -> None:
        print("(" , self.x, ";", self.y, ")", sep="")
```

Kỹ thuật Lập trình Python

35

35



Phương thức - Method

Bài tập

- Viết các phương thức get, set cho các thuộc tính lớp NhanVien



Class method

Giới thiệu

- **Class method** là những phương thức đặc trưng cho class và gắn liền với **class** thay vì với **object**.
- Loại method này chủ yếu dùng để xử lý dữ liệu cho các thuộc tính trong class.
- Cách quy định thêm **@classmethod** trước khi viết phương thức. Và thay **self** thành **cls**
- Truy cập phương thức bằng **tên class** hoặc qua tên **đối tượng**.



Class method

Ví dụ

```
class NhanVien:
    so_nv = 0
    def __init__(self, ma_nv):
        self.ma_nv = ma_nv
        NhanVien.so_nv += 1

    @classmethod
    def so_luong(cls):
        return cls.so_nv

n1 = NhanVien('121')
n2 = NhanVien('122')
n3 = NhanVien('123')

print('Số nhân viên:', NhanVien.so_luong()) #Truy cập method class qua tên class
print('Số nhân viên:', n1.so_luong()) #Truy cập method class qua tên đối tượng

print('Số nhân viên:', NhanVien.so_nv) #Truy cập thuộc tính lớp qua tên class
```

Kỹ thuật Lập trình Python

38

38



Static method

Static method

- **Static method** là những phương thức **tự do** đặt bên trong class, **không ràng buộc** về dữ liệu so với **class** hay **object**.
- Tạo static method trong class bằng cách dùng khai báo **@staticmethod** trước khi viết phương thức.
- Truy cập phương thức bằng **tên class** hoặc qua tên **đối tượng**

Kỹ thuật Lập trình Python

39

39



Static method

Ví dụ

```
class NhanVien:
    so_nv = 0
    def __init__(self, ma_nv):
        self.ma_nv = ma_nv
        NhanVien.so_nv += 1

    @classmethod
    def so_luong(cls):
        return cls.so_nv

    @staticmethod
    def tien_thuong(tien):
        return tien * 1.2

print('Tiền thưởng', NhanVien.tien_thuong(8_000_000))
```

Kỹ thuật Lập trình Python

40

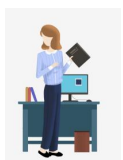
40



Khởi tạo đối tượng

Cú pháp

<ten đối tượng> = ClassName(...)



NhanVien
+ ma_nv
+ ho_ten
+ luong_cb
+ luong_ht
+ so_ng
+ inThongTin()
+ tinh_luong_ht()



```
class NhanVien:
    def __init__(self, maNV, hoTen, luongCB):
        self.maNV = maNV
        self.hoTen = hoTen
        self.luongCB = luongCB

    def xuat(self):
        print("\n")
        print("\t+Mã NV :", self.maNV)
        print("\t+Họ tên :", self.hoTen)
        print("\t+Lương CB:", self.luongCB)
```

constructor



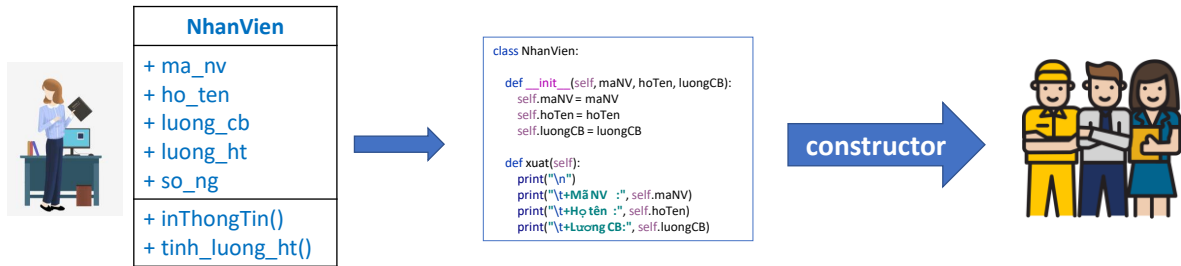
Ví dụ

```
nv1 = NhanVien(122, 'Nguyễn Văn A', 8_600_000, 22)
nv2 = NhanVien(123, 'Nguyễn Văn B', 6_500_000, 21)
nv3 = NhanVien(124, 'Nguyễn Văn C', 9_800_000, 18)
```

Kỹ thuật Lập trình Python

41

41



42

42



Tóm tắt

Lớp - Class

- Thuộc tính
 - Thuộc tính của đối tượng – Instance Attribute
 - Thuộc tính của lớp – Class Attribute
- Phương thức
 - Phương thức khởi tạo – Constructor
 - Phương thức của đối tượng – Instance method – Method
 - Phương thức của lớp – Class method
 - Phương thức tĩnh – Static method

43



BÀI TẬP ÁP DỤNG

1. Giải phương trình bậc nhất $aX + b = 0$
2. Giải phương trình bậc 2 $aX^2 + bX + c = 0$
3. Giải phương trình trùng phương $aX^4 + bX^2 + c = 0$

Code it!

Chú ý:

- Tất cả các bài được thiết kế theo phương pháp hướng đối tượng.



Hướng dẫn

Code it!

BacNhat
+ a
+ b
+ nghiem = None
+ __init__(a, b)
+ __str__()
+ in_pt()
+ giai_pt()



Bài tập QLVN: Áp dụng OOP (class - obj)

Viết chương trình tính tiền lương nhân viên (NV) cho một công ty. Công ty gồm 2 loại NV:

1. Nhân viên văn phòng

- Mã nhân viên
- Họ và tên
- Lương cơ bản
- Số ngày làm việc
- Lương hằng tháng

2. Nhân viên bán hàng

- Mã nhân viên
- Họ và tên
- Lương cơ bản
- Số sản phẩm bán được
- Lương hằng tháng

Lương hằng tháng tính bằng công thức:

- NV văn phòng = lương cơ bản + số ngày làm việc * 150_000đ
- NV bán hàng = lương cơ bản + số sản phẩm * 18_000đ

Chương trình đáp ứng các yêu cầu sau:

1. Khởi tạo dữ liệu các nhân viên
2. In các nhân viên: mã nhân viên, họ tên, lương cơ bản, **lương hằng tháng**
3. Tính lương các nhân viên
4. Tìm nhân viên theo mã nhân viên
5. Tìm lương hằng tháng cao nhất
6. Cập nhật lương cơ bản theo mã NV
7. Cập nhật số ngày làm của nhân viên văn phòng theo mã nhân viên.
8. Tính tổng tiền lương hằng tháng mà công ty phải trả cho các nhân viên.
9. Tìm nhân viên có lương cơ bản cao nhất
10. Tìm nhân viên bán hàng có lương hằng tháng thấp nhất
11. Sắp xếp NV giảm dần theo lương cơ bản.

Kỹ thuật Lập trình Python

46

46



Hướng dẫn

- Có bao nhiêu đối tượng tham gia vào bài toán?
- Số class?

Kỹ thuật Lập trình Python

47

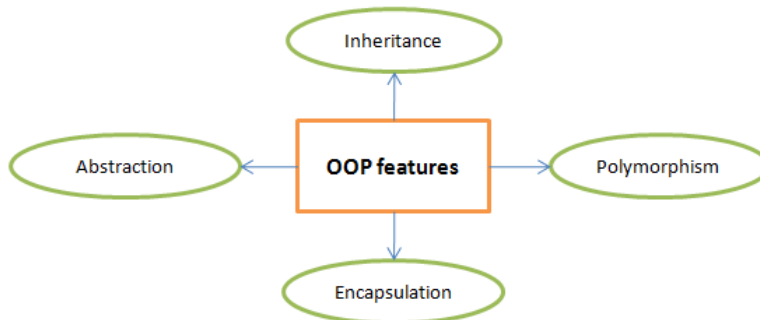
47



Đặc tính của OOP

Giới thiệu

- Các tính chất quan trọng ==> điểm mạnh của OOP



Tính kế thừa

Giới thiệu

- Kế thừa là sự thừa hưởng lại các thuộc tính, phương thức mà không cần khai báo lại các thuộc tính, phương thức.
- Kế thừa giúp tái sử dụng lại mã nguồn giúp lập trình viên dễ dàng rà soát bảo trì mã nguồn.

```

class A:
    def __init__(self, a):
        self.a = a

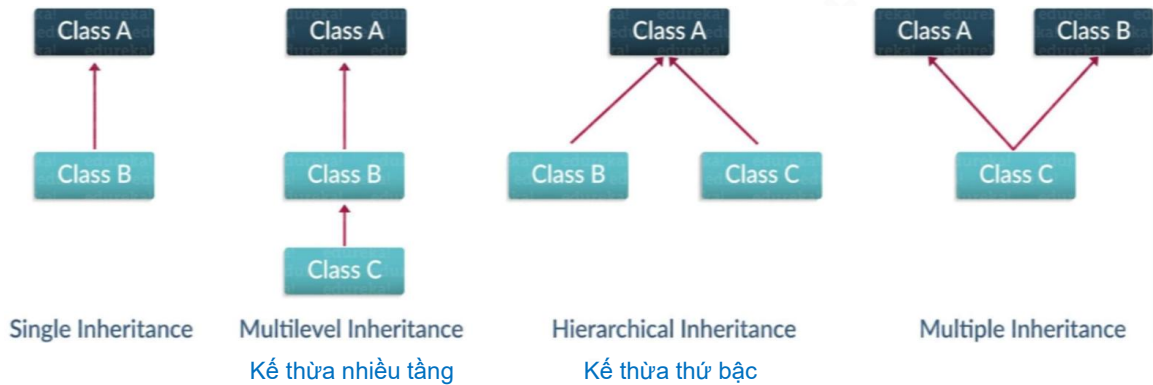
class B(A):
    def __init__(self, a, b):
        super().__init__(a)
        self.b = b

    def xuất(self):
        print(self.a, self.b)
  
```



Tính kế thừa

Các kiểu kế thừa



Kỹ thuật Lập trình Python

51

51



Tính kế thừa

Minh họa



```
class A:
    def __init__(self, a):
        self.a = a

class B(A):
    def __init__(self, a, b):
        super().__init__(a)
        self.b = b

    def xuat(self):
        print(self.a, self.b)
```

```
class A:
    def __init__(self, a):
        self.a = a

class B(A):
    def __init__(self, a, b):
        A.__init__(self, a)
        self.b = b

    def xuat(self):
        print(self.a, self.b)
```

Kỹ thuật Lập trình Python

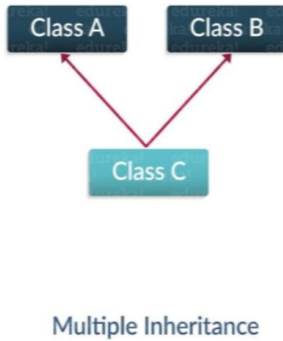
52

52



Tính kế thừa

Minh họa



```

class A:
    def __init__(self, a):
        self.a = a

class B:
    def __init__(self, b):
        self.b = b

class C(A, B):
    def __init__(self, a, b, c):
        A.__init__(self, a)
        B.__init__(self, b)
        self.c = c

    def xuất(self):
        print(self.a, self.b, self.c, end="\t")
  
```

Kỹ thuật Lập trình Python

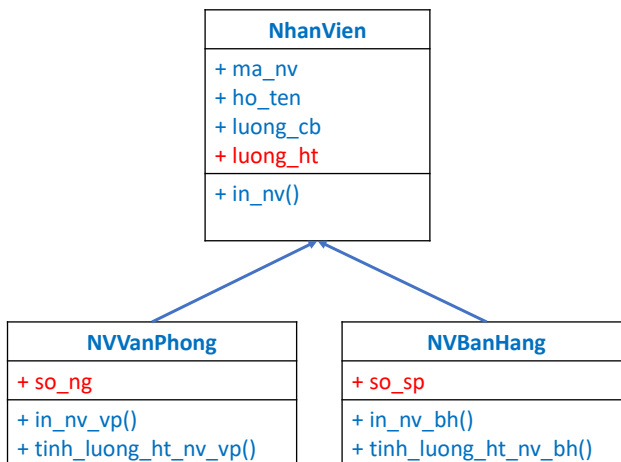
53

53



Tính kế thừa

Sơ đồ lớp



NVVanPhong
+ ma_nv + ho_ten + luong_cb + so_ng + luong_ht
+ in_nv_vp() + tinh_luong_ht_nv_vp()

NVBanHang
+ ma_nv + ho_ten + luong_cb + so_sp + luong_ht
+ in_nv_bh() + tinh_luong_ht_nv_bh()

Kỹ thuật Lập trình Python

54

54



Tính kế thừa

Python: Có nạp chồng phương thức?

- Trong các ngôn ngữ khác, nạp chồng là các phương thức có cùng tên, nhưng khác nhau về số lượng tham số/đối số truyền vào.
- Kỹ thuật nạp chồng làm tăng sự linh hoạt cho các phương thức.
- Python không hỗ trợ nạp chồng phương thức theo tham số.
- Xét ví dụ sau:

```
class A:
    def a(self, b):
        return b

    def a(self, b, c):
        return b + c

a = A()
print(a.a(1, 2))

print(a.a(1)) #Báo lỗi
```



Tính kế thừa

Để phương thức linh hoạt sử dụng

- Kỹ thuật tạo giá trị mặc nhiên

```
class A:
    def a(self, b=0, c=0):
        return b + c

a = A()

print(a.a(1, 2))
print(a.a(1))
print(a.a())
```



Tính kế thừa

Để phương thức linh hoạt sử dụng

- Kỹ thuật *args

```
class A:
    def a(self, *arg):
        return sum(arg)

a = A()

print(a.a(1, 2, 3))
print(a.a(1, 2))
print(a.a(1))
print(a.a())
```

Kỹ thuật Lập trình Python

59

59



Tính kế thừa

Để phương thức linh hoạt sử dụng

- Kỹ thuật **kwargs

```
class A:
    def a(self, **kwargs):
        return kwargs

obj = A()

print(obj.a(x=1))
print(obj.a(x=1, y=2))
print(obj.a(x=1, y=2, z=3))
```

Kỹ thuật Lập trình Python

60

60



Tính kế thừa

Ghi đè phương thức trong class con

• Ghi đè hoàn toàn:

- Là việc xóa bỏ và xây dựng lại toàn bộ nội dung của phương thức trong class cha.
- Khi nào phải ghi đè hoàn toàn?

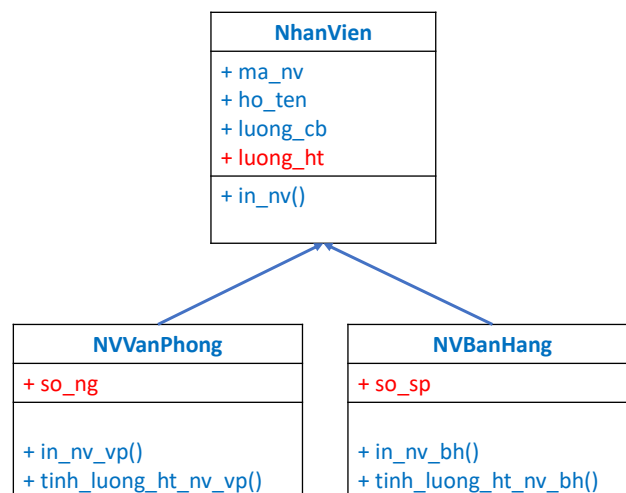
• Ghi đè không hoàn toàn:

- Là kế thừa lại một phần phương thức trong class cha và xây dựng thêm nội dung cho phương thức con.
- Sử dụng `super().tenPhuongThuc(...)`, `TenClassCha.tenPhuongThuc(self, ...)` để gọi đến phương thức trong class cha

Kỹ thuật Lập trình Python

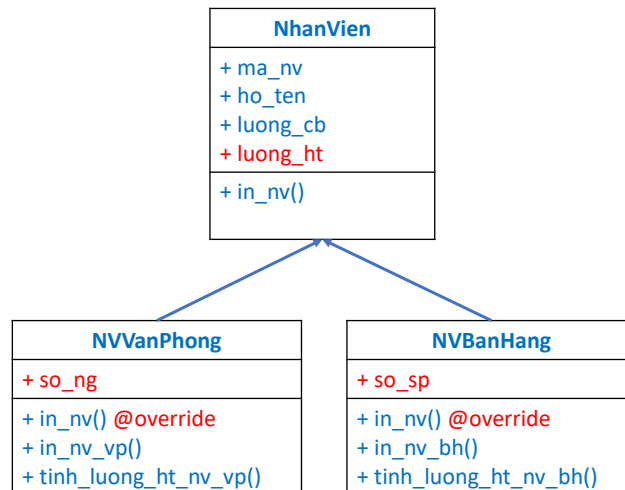
61

61



62

62



63

63



Tính kế thừa

Ghi đề phương thức trong class con

1. Một phương thức `a()` được xây dựng trong class cha (class A).
 2. Tiếp theo, class B kế thừa class A.
 3. Và trong class B, xây dựng thêm phương thức `a()` cùng tên.
- => Phương thức `a()` trong B sẽ ghi đề phương thức `a()` trong A

```

class A:
    def a(self):
        print("Phương thức a trong lớp A")

class B(A):
    def a(self):
        print("Phương thức a trong lớp B")

b = B()
b.a()
  
```

=> ghi đề hoàn toàn

64



Tính kế thừa

Ghi đè phương thức trong class con

```
class A:
    def a(self):
        print("Phương thức a trong lớp A")

class B(A):
    def a(self):
        super().a()
        print("Phương thức a trong lớp B")

b = B()
b.a()
```

=> ghi đè không hoàn toàn

Kỹ thuật Lập trình Python

65

65



Tính kế thừa

Minh họa ghi đè hoàn toàn

```
class A:
    def a(self, a=1, b=2):
        print('Class A:')
        print('\ta = ', a)
        print('\tb = ', b)

class B(A):
    def a(self):
        print('Class B')
```

```
a = A()

a.a(3, 4)
a.a(1, 2)
a.a(1)
a.a()

b = B()
b.a()
```

Kỹ thuật Lập trình Python

66

66



Tính kế thừa

Minh họa ghi đè không hoàn toàn

```
class A:
    def a(self, a=1, b=2):
        print('Class A:')
        print('\ta = ', a)
        print('\tb = ', b)

class B(A):
    def a(self):
        print('Class B')
        super().a()
```

```
a = A()
a.a(3, 4)
a.a(1, 2)
a.a(1)
a.a()

b = B()
b.a()
```

Kỹ thuật Lập trình Python

67

67



Tính đóng gói

Khái niệm

- Tính đóng gói (encapsulation) trong OOP để ẩn thông tin và hành vi của đối tượng bên trong lớp.
- Để bảo vệ thuộc tính và các phương thức của lớp, ngăn không cho truy cập trực tiếp và sửa đổi.

Lợi ích của tính đóng gói

- Bảo vệ dữ liệu.
- Ứng dụng nguyên tắc trừu tượng.
- Quản lý sự thay đổi.
- Cung cấp giao diện rõ ràng.
- Tăng tính bảo mật và kiểm soát truy cập.

```
nv1 = NhanVien(122, 'Nguyễn Văn A', 8_600_000)
nv2 = NhanVien(123, 'Nguyễn Văn B', 6_500_000)
nv3 = NhanVien(124, 'Nguyễn Văn C', 9_800_000)
```

Kỹ thuật Lập trình Python

68

68



Phạm vi truy cập - Access modifiers

Phạm vi truy cập?

- Là việc quy định **phạm vi**, **quyền hạn** của thuộc tính và phương thức.

Ví dụ

- Một đối tượng A được quy định **phạm vi** về **thuộc tính** và **phương thức**
- Thì các đối tượng khác sẽ có mức độ truy cập vào đối tượng A là khác nhau.

A
- x # y + z
- method_x() # method_y() + method_z()

Kỹ thuật Lập trình Python

69

69



Phạm vi truy cập - Access modifiers

- Khi khai báo/xây dựng một class có thể dùng các **ký pháp** để thể hiện **phạm vi truy cập** của thuộc tính, phương thức...

- Phạm vi truy cập:

- Public: để trống
- Protected: ký pháp `_`
- Private: ký pháp `__`

- Phạm vi truy cập sử dụng để thể hiện mức độ riêng tư trong khai báo thuộc tính, constructor, phương thức...

```
class NhanVien:
    def __init__(self, ma_nv, ho_ten, luong_cb):
        self.ma_nv = ma_nv
        self._ho_ten = ho_ten
        self.__luong_cb = luong_cb

    def xuất(self):
        print("\n")
        print("\t+Mã NV :", self.ma_nv)
        print("\t+Họ tên :", self._ho_ten)
        print("\t+Lương CB:", self.__luong_cb)
```

Kỹ thuật Lập trình Python

70

70



Phạm vi truy cập - Access modifiers

Mô tả

Quyền hạn - Phạm vi	Mô tả
protected	- Truy cập từ bên trong lớp và các lớp con kế thừa của lớp đó. - Có thể truy cập bên ngoài lớp.
private	- Không thể truy cập bên ngoài lớp. - Không thể truy cập từ các lớp con kế thừa của lớp đó. - Truy cập từ bên trong lớp.
public	Truy cập mọi phạm vi.

Kỹ thuật Lập trình Python

71

71



Phạm vi truy cập - Access modifiers

Cú pháp khai báo

- Khai báo quyền hạn cho thuộc tính, phương thức.

Quyền hạn – Phạm vi	Phương pháp	Ví dụ
protected	—	self._maNV def _tinhLuong(self, ...)
private	--	self.__maNV def __tinhLuong(self, ...)
public	Để trống	self.maNV def tinhLuong(self, ...)

Kỹ thuật Lập trình Python

72

72



Phạm vi truy cập - Access modifiers

Bảng tóm tắt quyền hạn và phạm vi

	Internal class	Kế thừa	Trong cùng package	Khác package
Private --	☒			
Protected _	☒	☒	☒	☒
Public (Để trống)	☒	☒	☒	☒



Phạm vi truy cập - Access modifiers

Ví dụ minh họa

- Xem demo GV



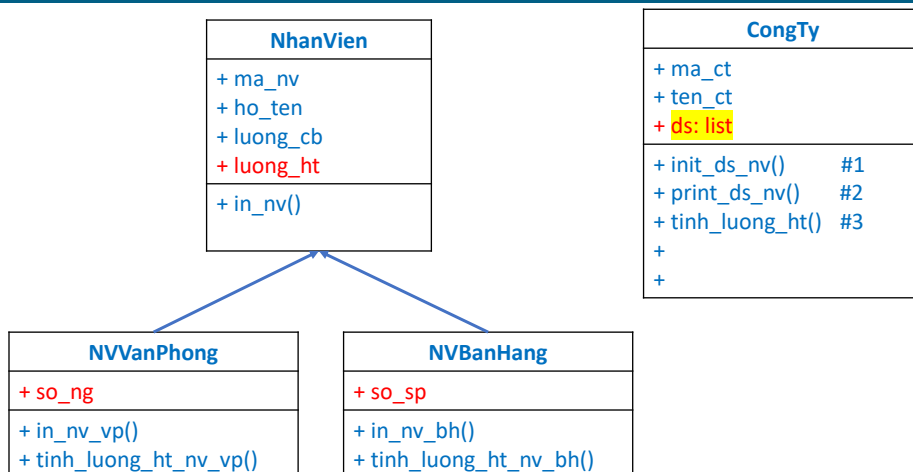
Khuyến nghị

Khuyến nghị khi khai báo quyền hạn cho thuộc tính, phương thức

- **Đối với thuộc tính** (nên khai báo 2 quyền)
 - **__private**: nếu thuộc tính chỉ dùng riêng trong một class/đối tượng
 - **__protected**: nếu thuộc tính đó được các đối tượng con kế thừa
- **Đối với phương thức** (tùy thuộc vào phạm vi mà có thể khai báo quyền)
 - **__private**: nếu các phương thức chỉ sử dụng trong nội bộ.
 - **__protected**: nếu các phương thức được sử dụng trong đối tượng con.
 - **public**: nếu các phương thức được sử dụng ở mọi nơi, kể cả việc khác package.

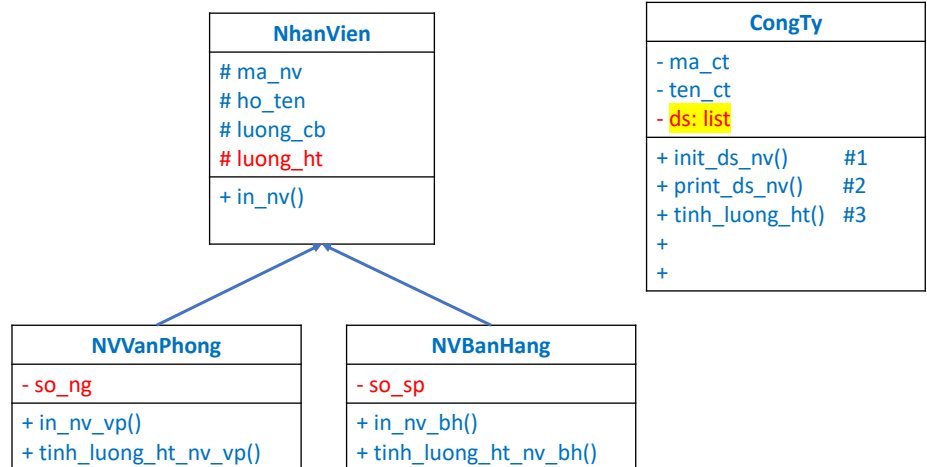


QLNV: Chưa đóng gói





QLNV: Áp dụng tính đóng gói



Kỹ thuật Lập trình Python

78

78



get - Lấy thuộc tính

Lấy thuộc tính của đối tượng

- Khai báo quyền hạn thuộc tính là **private** thì bên ngoài sẽ không truy cập được thuộc tính.
- Trong trường hợp này, muốn lấy thông tin thuộc tính (private) của class thì phải **xây dựng thêm phương thức truy vấn **get_ten...(self)**** để lấy giá trị thuộc tính.
- Tham khảo cách sau:

```

def get_ten_thuoc_tinh(self):
    return self.__ten_thuoc_tinh

@property
def name(self):
    return self.__name
  
```

Kỹ thuật Lập trình Python

79

79



get - Lấy thuộc tính

Ví dụ

```
class A:
    def __init__(self, value):
        self.__value = value

    def getValue(self):
        return self.__value

    @property
    def value(self):
        return self.__value
```

```
a = A(39)
print(a.value)
print(a.getValue())
```

Kỹ thuật Lập trình Python

80

80



get - Lấy thuộc tính

Ví dụ

```
class NhanVien:
    def __init__(self, name):
        self.__name = name

    def getName(self):
        return self.__name

    @property
    def name(self):
        return self.__name
```

```
n1 = NhanVien('Văn A')
n2 = NhanVien('Văn B')

print('Tên nhân viên n2:', n2.name)
print('Tên nhân viên n1:', n1.getName())
```

Kỹ thuật Lập trình Python

81

81



set - Cập nhật thuộc tính

Sửa thuộc tính của đối tượng

- Khi khai báo quyền hạn thuộc tính là `private` thì tại ngoài class sẽ không truy cập được thuộc tính.
- Muốn **cập nhật** giá trị thuộc tính (private) của class thì phải xây dựng thêm phương thức **set_ten...()** để **cập nhật** giá trị thuộc tính.
- Tham khảo cách sau:

```
def set_value(self, value):
    self.__value = value
```

```
@property
def value(self):
    return self.__value
```

```
@property
def value(self):
    return self.__value
```

- Chú ý: việc thiết kế phương thức set không được tùy tiện.

Kỹ thuật Lập trình Python

82

82



set - Cập nhật thuộc tính

Ví dụ

```
class C:
    def __init__(self, value):
        self.__value = value

    def setValue(self, value):
        self.__value = value

    @property
    def value(self):
        return self.__value

    @value.setter
    def value(self, valueNew):
        self.__value = valueNew
```

```
a = C(39)
print(a.value)

a.value = 79
print(a.value)

a.setValue(99)
print(a.value)
```

Kỹ thuật Lập trình Python

83

83



set - Cập nhật thuộc tính

Ví dụ

```
class NhanVien:
    def __init__(self, name):
        self.__name = name

    @property
    def name(self):
        return self.__name

    @name.setter
    def name(self, nameNew):
        self.__name = nameNew

    def setName(self, nameNew):
        self.__name = nameNew
```

```
n1 = NhanVien('Văn A')
n2 = NhanVien('Văn B')
print('Tên nhân viên n1:', n1.name)
print('Tên nhân viên n2:', n2.name)

n1.name = 'Văn C'
print('Tên nhân viên n1:', n1.name)

n2.setName('Văn D')
print('Tên nhân viên n2:', n2.name)
```

Kỹ thuật Lập trình Python

84

84



Decorator trong Python

Decorator?

- Là một kỹ thuật thiết kế mẫu giúp đối tượng trở nên linh hoạt.

Built-in Decorators

- Python cung cấp các decorator có sẵn để hỗ trợ định nghĩa property, class method, static method... cho class.

Decorator	Mô tả
@property	Định nghĩa phương thức để get và set các property.
@classmethod	Định nghĩa phương thức class method gọi bằng tên class
@staticmethod	Định nghĩa các static method.

Kỹ thuật Lập trình Python

85

85



@property

```
class NhanVien:
    def __init__(self, name):
        self.__name = name

    @property
    def name(self):
        return self.__name

    @name.setter
    def name(self, value):
        self.__name = value

    @name.deleter # property-name.deleter decorator
    def name(self, value):
        print('Đang xóa...', self.__name)
        del self.__name
        print('Đã xóa.')
```

```
n1 = NhanVien('Văn A')
n2 = NhanVien('Văn B')

print('Tên nhân viên n2:', n2.name)
n1.name = 'Văn C'
print('Tên nhân viên n1:', n1.name)
```

Kỹ thuật Lập trình Python

86

86



Bài tập QLNV: Áp dụng OOP (Tính đóng gói, kế thừa)

Viết chương trình tính tiền lương nhân viên (NV) cho một công ty. Công ty gồm 2 loại NV:

1. **Nhân viên văn phòng**
 - Mã nhân viên
 - Họ và tên
 - Lương cơ bản
 - Số ngày làm việc
 - Lương hằng tháng
2. **Nhân viên bán hàng**
 - Mã nhân viên
 - Họ và tên
 - Lương cơ bản
 - Số sản phẩm bán được
 - Lương hằng tháng

Lương hằng tháng tính bằng công thức:

- NV văn phòng = lương cơ bản + số ngày làm việc * 150_000đ
- NV bán hàng = lương cơ bản + số sản phẩm * 18_000đ

Chương trình đáp ứng các yêu cầu sau:

1. Khởi tạo dữ liệu các nhân viên
2. In các nhân viên: mã nhân viên, họ tên, lương cơ bản, lương hằng tháng
3. Tính lương các nhân viên
4. Tìm nhân viên theo mã nhân viên
5. Tìm lương hằng tháng cao nhất
6. Cập nhật lương cơ bản theo mã NV
7. Cập nhật số ngày làm của nhân viên văn phòng theo mã nhân viên.
8. Tính tổng tiền lương hằng tháng mà công ty phải trả cho các nhân viên.
9. Tìm nhân viên có lương cơ bản cao nhất
10. Tìm nhân viên bán hàng có lương hằng tháng thấp nhất
11. Sắp xếp NV giảm dần theo lương cơ bản.

Kỹ thuật Lập trình Python

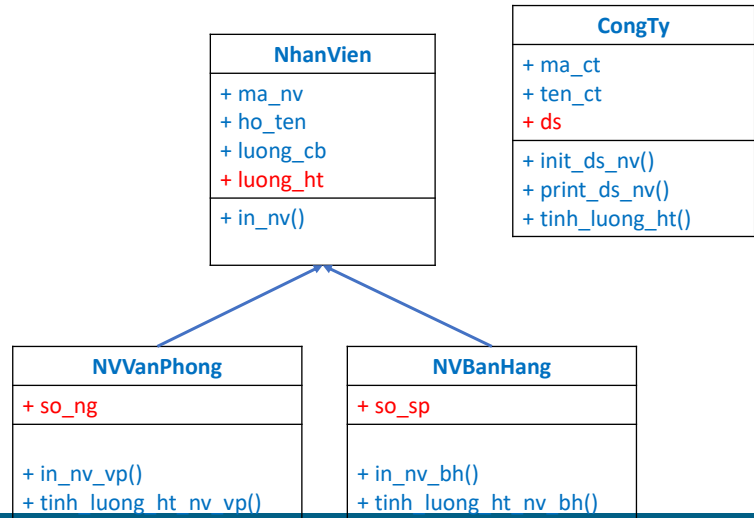
87

87



Hướng dẫn 1

- Đã áp dụng:
 - Tính kế thừa



Kỹ thuật Lập trình Python

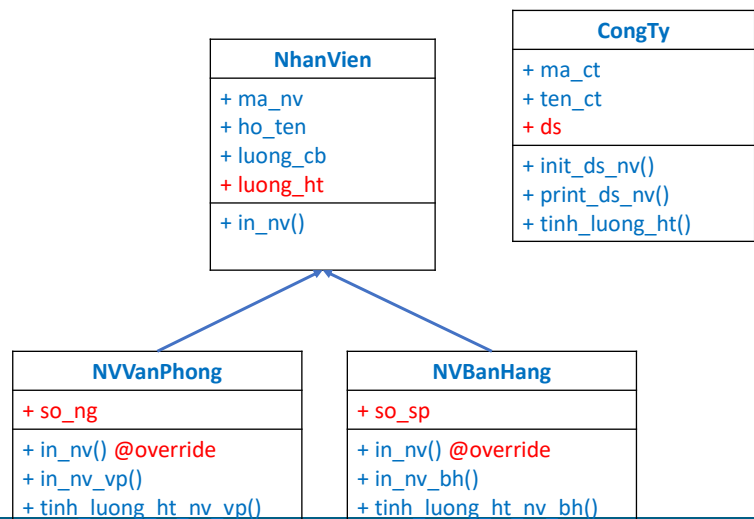
88

88



Hướng dẫn 2

- Đã áp dụng:
 - Tính kế thừa
 - Ghi đè trong kế thừa



Kỹ thuật Lập trình Python

89

89



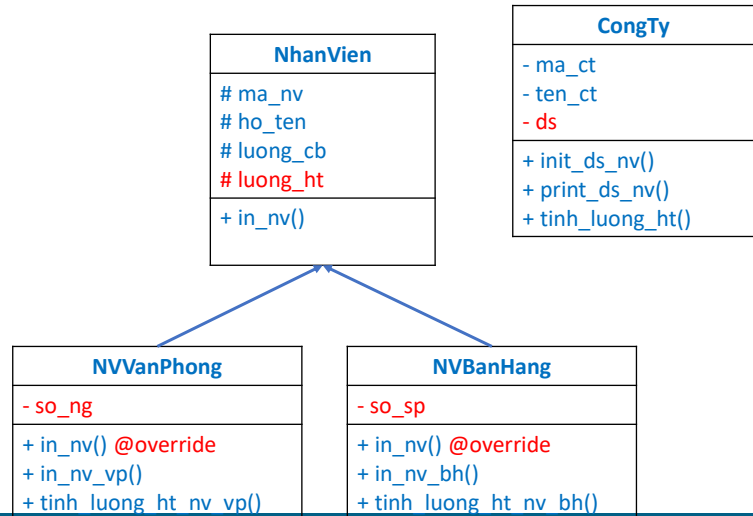
Hướng dẫn 3

• Đã áp dụng:

- Tính kế thừa
- Ghi đè trong kế thừa
- Đóng gói (3)

Chú ý khi vẽ sơ đồ class

- private
- # protected
- + public



Kỹ thuật Lập trình Python

90

90



Tính đa hình

Giới thiệu

- Đa hình trong OOP cho phép các đối tượng của các lớp khác nhau => có các phương thức cùng tên với nhau nhưng thực hiện xử lý khác nhau.
- Đa hình trong Python thường được thực hiện thông qua việc sử dụng kế thừa, ghi đè phương thức và sử dụng các phương thức có sẵn.
- Vậy vậy, tính đa hình được thể hiện thông qua tính kế thừa và ghi đè phương thức.

Ý nghĩa của tính đa hình

- Kiểm soát các phương thức dùng chung của các lớp giống hoặc khác nhau về hành vi bản chất.

Kỹ thuật Lập trình Python

91

91



Tính đa hình

Đa hình trong kế thừa

- Các lớp con có thể kế thừa các phương thức từ lớp cha và có thể ghi đè bằng cách định nghĩa lại phương thức của lớp con.
- Khi tạo ra một đối tượng rồi gọi phương thức từ đối tượng vừa tạo:
 - Phương thức của lớp con sẽ được gọi nếu đã được định nghĩa trong lớp con.
 - Ngược lại phương thức của lớp cha sẽ được gọi.

```
if __name__ == '__main__':
    b = B(1, 2)
    b.methodA()

    c = C(3, 4)
    c.methodA()
```

```
class A:
    def __init__(self, a):
        self._a = a

    def methodA(self):
        print('Class A. a = ', self._a)
```

```
class B(A):
    def __init__(self, a, b):
        super().__init__(a)
        self._b = b

    def methodA(self):
        print('Class B. a = ', self._a, 'b = ', self._b)
```

```
class C(A):
    def __init__(self, a, b):
        super().__init__(a)
        self._b = b
```

Kỹ thuật Lập trình Python

92

92



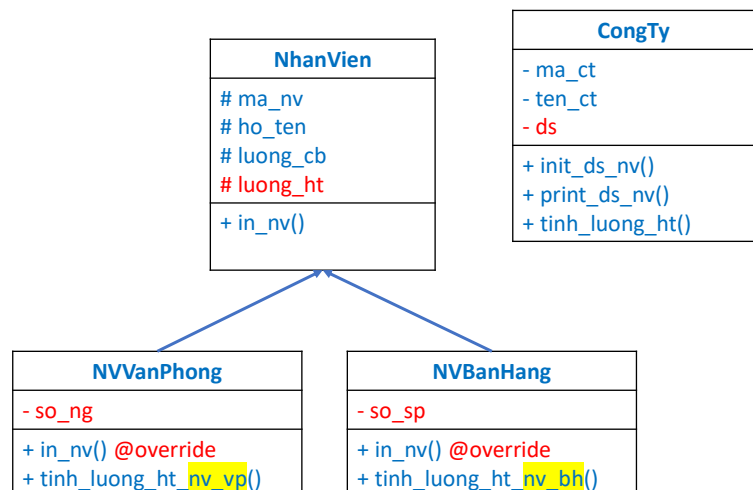
Hướng dẫn 3

• Đã áp dụng:

- Tính kế thừa
- Ghi đè trong kế thừa
- Đóng gói (3)

Chú ý khi vẽ sơ đồ class

- private
- # protected
- + public



Kỹ thuật Lập trình Python

93

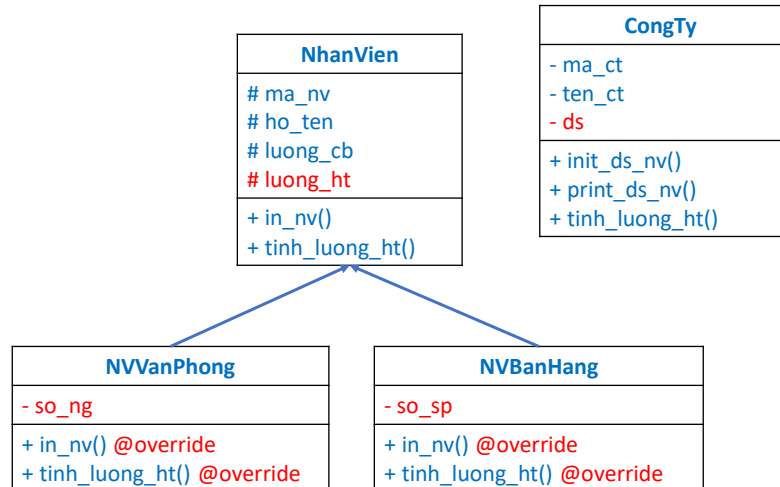
93



Hướng dẫn 4

- Đã áp dụng:

- Tính kế thừa
- Ghi đè trong kế thừa
- Đóng gói (3)
- Tính đa hình (4)



Kỹ thuật Lập trình Python

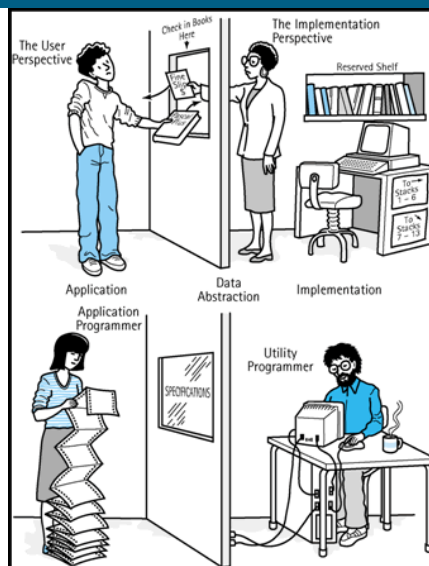
94

94



Tính trừu tượng

Ý tưởng?



Hình (Internet)

Kỹ thuật Lập trình Python

106

106



Tính trừu tượng

Tính trừu tượng?

- Là tính chất không thể hiện cụ thể mà chỉ nêu tên vấn đề.
- Cho phép phớt lờ, không chú ý đến các đặc điểm của đối tượng. Thay vào đó, chỉ tập trung vào các đặc điểm cốt lõi của đối tượng mà bài toán đề cập.

Ưu điểm

- Tập trung vào những thông tin cốt lõi của đối tượng, thay vì quan tâm đến cách nó thực hiện
- Tính trừu tượng cung cấp nhiều chức năng mở rộng thêm nếu kết hợp với **tính đa hình** và **kế thừa**.

Kỹ thuật Lập trình Python

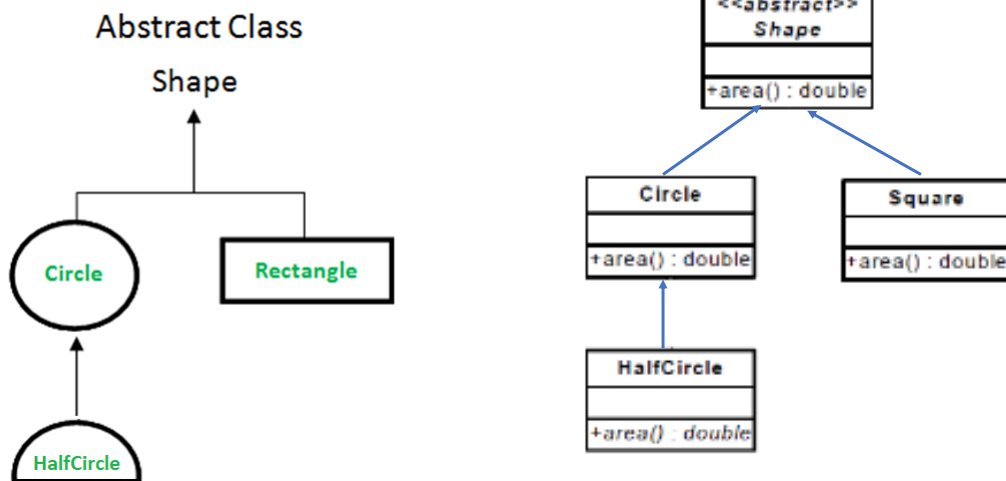
108

108



Tính trừu tượng

Phương pháp



Kỹ thuật Lập trình Python

109

109



Tính trừu tượng

Phương pháp

- Sử dụng module **abc** để:
 - Kế thừa lớp trừu tượng **ABC** - Abstract Base Classes
 - Decorator phương thức **abstractmethod**, hay khai báo ra các phương thức trừu tượng.
- Abstract class không phải là một class cụ thể, không thể khởi tạo, là một bản thiết kế cho các class khác.
- Đóng vai trò như một cái sườn để tạo ra các lớp con kế thừa từ cái sườn này.

```
from abc import ABC, abstractmethod
```

```
class TruuTuong(ABC):
```

```
    @abstractmethod
```

```
    def methodA(self):
```

```
        pass
```

```
    @abstractmethod
```

```
    def methodB(self):
```

```
        pass
```

```
# Tạo các lớp kế thừa từ lớp trừu tượng để triển khai
```

```
class TrienKhai(TruuTuong):
```

```
    def methodA(self):
```

```
        print('Triển khai chi tiết A')
```

```
    def methodB(self):
```

```
        print('Triển khai chi tiết B')
```

Kỹ thuật Lập trình Python

110

110



Tính trừu tượng

Phương thức trừu tượng (Abstract method)

- Sử dụng từ khóa **@abstractmethod** khi khai báo tên phương thức.
- Chỉ cần khai báo phương thức, không cần triển khai phương thức.

Tham khảo

```
@abstractmethod
```

```
def methodA(self):
```

```
    pass
```

```
@abstractmethod
```

```
def methodB(self):
```

```
    pass
```

Chú ý:

- Quyền hạn không để **private**
- Class kế thừa triển khai **override** phương thức trừu tượng này.

Kỹ thuật Lập trình Python

111

111



Bài tập QLNV: Áp dụng OOP (Tính đóng gói, kế thừa, trừu tượng)

Viết chương trình tính tiền lương nhân viên (NV) cho một công ty. Công ty gồm 2 loại NV:

1. Nhân viên văn phòng

- Mã nhân viên
- Họ và tên
- Lương cơ bản
- Số ngày làm việc
- Lương hằng tháng

2. Nhân viên bán hàng

- Mã nhân viên
- Họ và tên
- Lương cơ bản
- Số sản phẩm bán được
- Lương hằng tháng

Lương hằng tháng tính bằng công thức:

- NV văn phòng = lương cơ bản + số ngày làm việc * 150_000đ
- NV bán hàng = lương cơ bản + số sản phẩm * 18_000đ

Chương trình đáp ứng các yêu cầu sau:

1. Khởi tạo dữ liệu các nhân viên
2. In các nhân viên: mã nhân viên, họ tên, lương cơ bản, lương hằng tháng
3. Tính lương các nhân viên
4. Tìm nhân viên theo mã nhân viên
5. Tìm lương hằng tháng cao nhất
6. Cập nhật lương cơ bản theo mã NV
7. Cập nhật số ngày làm của nhân viên văn phòng theo mã nhân viên.
8. Tính tổng tiền lương hằng tháng mà công ty phải trả cho các nhân viên.
9. Tìm nhân viên có lương cơ bản cao nhất
10. Tìm nhân viên bán hàng có lương hằng tháng thấp nhất
11. Sắp xếp NV giảm dần theo lương cơ bản.

Kỹ thuật Lập trình Python

112

112



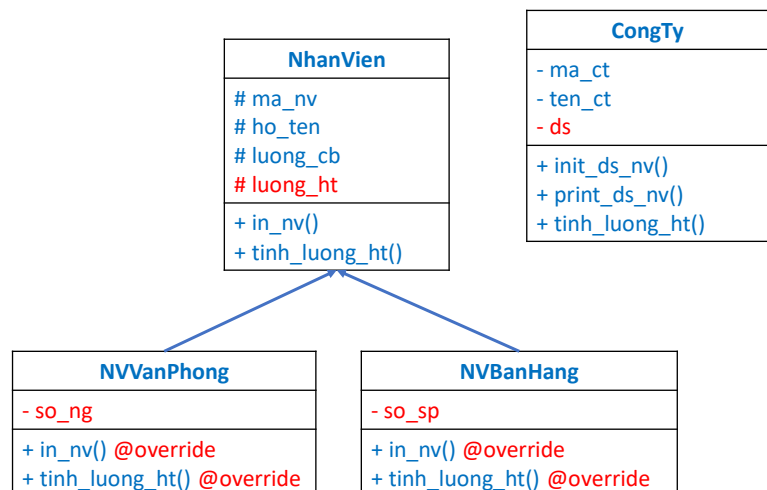
Hướng dẫn 3

• Đã áp dụng:

- Tính kế thừa
- Ghi đè trong kế thừa
- Đóng gói (3)

Chú ý khi vẽ sơ đồ class

- private
- # protected
- + public



Kỹ thuật Lập trình Python

113

113



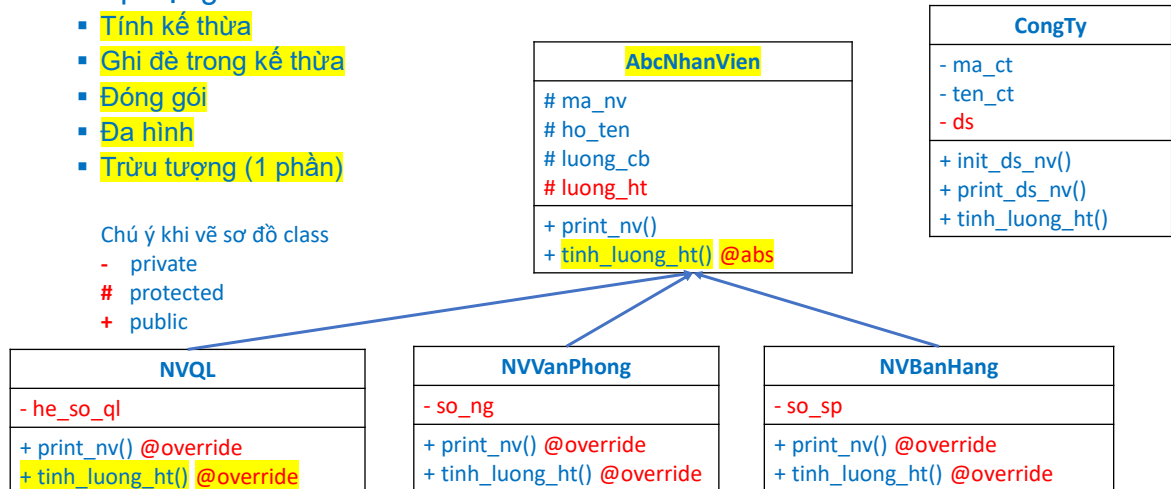
Hướng dẫn 5 (1 phần)

• Đã áp dụng:

- Tính kế thừa
- Ghi đè trong kế thừa
- Đóng gói
- Đa hình
- Trừu tượng (1 phần)

Chú ý khi vẽ sơ đồ class

- private
- # protected
- + public



Kỹ thuật Lập trình Python

114

114



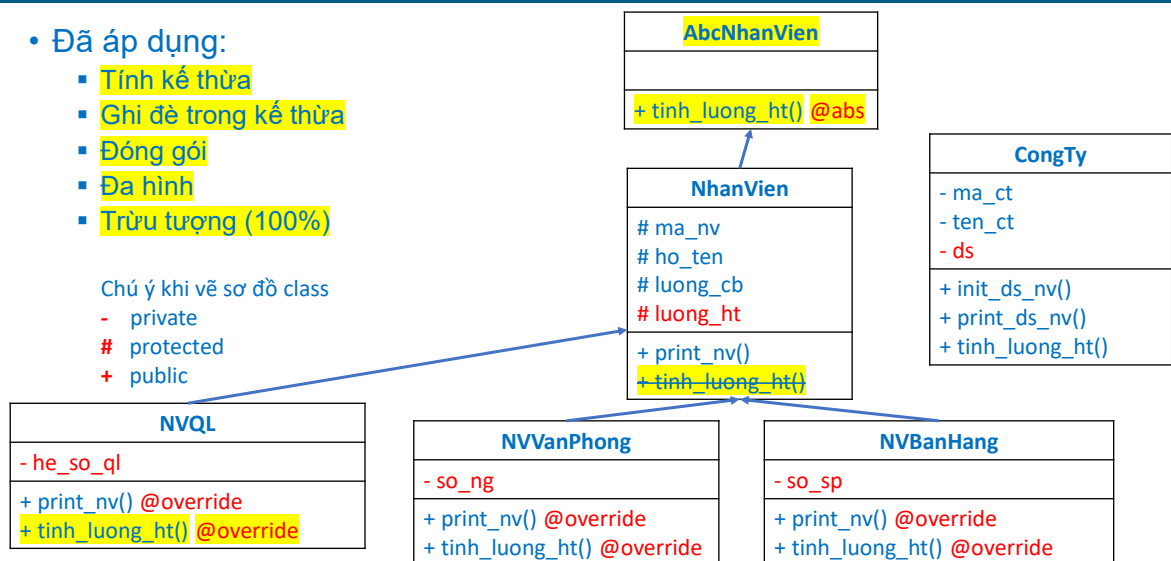
Hướng dẫn 6 (100% trừu tượng)

• Đã áp dụng:

- Tính kế thừa
- Ghi đè trong kế thừa
- Đóng gói
- Đa hình
- Trừu tượng (100%)

Chú ý khi vẽ sơ đồ class

- private
- # protected
- + public



Kỹ thuật Lập trình Python

115

115



Nâng cấp

Công ty xuất hiện thêm 1 đối tượng **nhân viên quản lý**. Nhân viên quản lý có các thông tin sau:

- Mã nhân viên
- Họ và tên
- Lương cơ bản
- Số ngày làm
- Hệ số trách nhiệm
- Lương hằng tháng = lương cơ bản + số ngày làm * 250_000 * hệ số trách nhiệm. Nếu hệ số trách nhiệm > 3.5 thì thưởng thêm 20%.
- Yêu cầu: Thiết kế và bổ sung class thích hợp hạn chế sửa code cũ.

Kỹ thuật Lập trình Python

120

120

Viết chương trình tính tiền lương nhân viên (NV) cho một công ty. Công ty gồm 2 loại NV:

1. Nhân viên văn phòng

- Mã nhân viên
- Họ và tên
- Lương cơ bản
- Số ngày làm việc
- Lương hằng tháng

2. Nhân viên bán hàng

- Mã nhân viên
- Họ và tên
- Lương cơ bản
- Số sản phẩm bán được
- Lương hằng tháng

Lương hằng tháng tính bằng công thức:

- NV văn phòng = lương cơ bản + **số ngày** làm việc * 150_000đ
- NV bán hàng = lương cơ bản + **số sản phẩm** * 18_000đ

Chương trình đáp ứng các yêu cầu sau:

1. Khởi tạo dữ liệu các nhân viên
2. In các nhân viên: mã nhân viên, họ tên, lương cơ bản, **lương hằng tháng**
3. Tính lương các nhân viên
4. Tìm nhân viên theo mã nhân viên
5. Tìm lương hằng tháng cao nhất
6. Cập nhật lương cơ bản theo mã NV
7. Cập nhật số ngày làm của nhân viên văn phòng theo mã nhân viên.
8. Tính tổng tiền lương hằng tháng mà công ty phải trả cho các nhân viên.
9. Tìm nhân viên có lương cơ bản cao nhất
10. Tìm nhân viên bán hàng có lương hằng tháng thấp nhất
11. Sắp xếp NV giảm dần theo lương cơ bản.

124



128



Hỏi - Đáp



129